CHENNAI RELIEF CONTROL

AN EXPLAINABLE RELIEF PLANNER FOR CHENNAI DISASTERS USING A* ROUTING AND A GENETIC ALGORITHM

A PROJECT REPORT

for

CS23533 FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Submitted by

240701212 – JENELL SG

240701327 – KANISHKA S

III YEAR B.E.

in

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

THANDALAM, CHENNAI 602 105

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**DISASTER RELIEF CONTROL: AN EXPLAINABLE RELIEF PLANNER FOR CHENNAI DISASTERS USING A* ROUTING AND A GENETIC ALGORITHM**" is the bonafide work of "**JENELL SG (240701212) and KANISHKA S (240701227)**" who carried out the project work under my supervision.

Student Signature with Name

1. JENELL SG
2. KANISHKA S

Submitted for the Practical Examination held on _____

SIGNATURE OF THE FACULTY INCHARGE

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
1	INTRODUCTION	1
2	LITERATURE REVIEW	4
3	PROPOSED SYSTEM	7
	3.1 ARCHITECTURE DIAGRAM	8
4	MODULES DESCRIPTION	10
	4.1 MODULE 1: HAZARD MODEL AND NEEDS	10
	4.2 MODULE 2: ROAD NETWORK AND A* ROUTING	11
	4.3 MODULE 3: DISPATCH BY GENETIC ALGORITHM	13
	4.4 MODULE 4: WEB APPLICATION	15
5	IMPLEMENTATION AND RESULTS	17
	5.1 EXPERIMENTAL SETUP	17
	5.2 RESULTS	18
6	CONCLUSION AND FUTURE WORK	29
7	REFERENCES	32

ABSTRACT

When rain, wind or an earthquake strikes Chennai, a relief coordinator has to decide which neighbourhoods need help, which roads are still open and which truck should carry what to where. This project, **Chennai Relief Control**, is a web application that makes these decisions for 19 zones and 23 depots using classical AI only, so that every number traces back to a written rule.

Live or replayed weather and earthquake readings are turned into a 0 to 100 disaster meter for each zone and a list of areas that need food, water, medicine, blankets and kits. Floods slow and close roads on an OpenStreetMap network of 8,044 junctions. A* finds the fastest open route, and a genetic algorithm assigns 46 trucks to areas and orders their visits, giving every delivery a clock-time arrival.

The planner was tested on four past events and two drills. A* matched an independent Dijkstra search on 1,008 routes. The tests also found two faults: a clamped score makes the shipped genetic algorithm return its first plan, and the Dispatch screen counts cut-off areas as reached, showing 67% delivered for the 2015 floods when 43% is reachable by road. Removing one line lifts road coverage to 64.8% and beats three simple baselines on five of six events. Depots and stock are placeholders.

CHAPTER 1
INTRODUCTION

In nine years Chennai has been hit by a flood, a wind storm and two rain-heavy cyclones, and each one stressed the city in a different way (Table 1.1). Every time, relief teams faced the same question: which depot sends what to which neighbourhood, and along which roads that are still open.

Table 1.1 Four events and the replay input used for each

Event	What was reported	Replay input (24 h rain, peak gust)
Floods, Dec 2015	Tambaram recorded 490 mm in 24 hours on 1 December; Chennai received 1,049 mm in November, the most since 1918; 422 deaths officially counted in Tamil Nadu; over 1.8 million people displaced (Wikipedia, 2026a)	345 mm, 45 km/h
Cyclone Vardah, Dec 2016	Crossed the coast close to Chennai on 12 December; about 1.5 million trees were uprooted in Chennai and its suburbs (Wikipedia, 2026b)	90 mm, 140 km/h
Cyclone Michaung, Dec 2023	530 mm at Nungambakkam over 2 to 4 December; the airport closed on 4 December; at least 17 deaths in the storm overall (Wikipedia, 2026c)	290 mm, 95 km/h
Cyclone Fengal, Nov 2024	Landfall near Puducherry on 30 November; up to 130 mm of rain in Chennai (Wikipedia, 2026d)	140 mm, 80 km/h

The replay inputs are representative round figures chosen for each event, not official measurements. They sit in one editable file, so a different reading of an event can be tried in minutes. The 2015 flood also shows why routes matter as much as stock. On 1 December the outflow from Chembarambakkam lake was raised from 12,000 to 20,960 cusecs, and a later release of 29,000 cusecs ran for 21 hours into the Adyar (The News Minute, n.d.). A release of that size can close roads along a river within hours, which is the situation the road model in this project is built for.

Problem Statement

Planning relief for such an event is hard for three reasons, and this project addresses each one.

- **The danger changes by neighbourhood and by hour.** A single city-wide alert says little about whether Velachery or Manali needs help first.
- **The road network changes during the event.** The fastest route on a dry day may cross a flooded underpass.
- **Trucks, stock and time are limited.** More areas ask for help than one fleet can reach in the first hours, so the order of visits decides who is helped early.

Objectives

1. To score the danger in each of 19 zones from live or replayed rain, wind and shaking, using written rules that a person can audit.
2. To derive which neighbourhoods need relief and how much food, water, medicine, blankets and emergency kits each needs.
3. To route trucks over the real road network with A*, while floods slow or close roads.
4. To assign trucks to areas and order the visits with a genetic algorithm, giving every delivery an arrival time in clock time.
5. To present all of it in a web application that works on a phone and on a desktop.
6. To evaluate the planner honestly: check A* against an independent search, and compare the genetic algorithm with simple baselines on six events.

Scope

Covered. Chennai's 15 corporation zones plus Avadi, Pallavaram, Chrompet and Tambaram (19 zones, 712 km², 6.48 million people by the project's modelled estimate); 57 recorded flood hotspots; 23 depots and 46 trucks; and one planning round from a chosen start time. The planner uses classical AI only, with no machine learning, so every number traces back to a rule.

Not covered. Live traffic, multi-trip schedules, rescue of people (the plan moves supplies only), accounts and permissions, and any learned forecasting model.

Not claimed. The 23 depots, 46 trucks and their stock are placeholders, because Chennai does not publish depot addresses as open data. The replay inputs are representative figures, and the zone populations are modelled estimates. The results therefore measure how well the planner works on a realistic stand-in, not how relief would go in a real emergency.

CHAPTER 2
LITERATURE REVIEW

Disaster logistics is a well-studied field, and Altay and Green (2006) survey it. The algorithms used in this project are established ones, so the review asks what earlier work offers and where it falls short for a small, open, explainable relief planner. Six works that shaped the design are reviewed below, each with its methodology and its main drawback.

1. Dantzig and Ramser (1959): "The Truck Dispatching Problem"

Methodology: Formulates the problem of serving many stops from one depot with a capacity-limited fleet, and proposes a way to build short routes.

Drawback: One depot, no hazards and no priority between stops.

2. Hart, Nilsson and Raphael (1968): "A Formal Basis for the Heuristic Determination of Minimum Cost Paths"

Methodology: Introduces A*, a best-first search that orders junctions by the cost so far plus a heuristic estimate, and shows that it returns a cheapest path when the estimate never overestimates the remaining cost.

Drawback: It needs a cost model; on its own it knows nothing about flooded or closed roads.

3. Haghani and Oh (1996): "Formulation and Solution of a Multi-Commodity, Multi-Modal Network Flow Model for Disaster Relief Operations"

Methodology: Models the movement of several relief commodities by several modes of transport as a network flow problem, solved with heuristics.

Drawback: Needs a solver and fixed demands, and takes no live hazard input.

4. Baker and Ayechew (2003); Prins (2004): genetic and evolutionary algorithms for vehicle routing

Methodology: Apply genetic and evolutionary search to the capacitated vehicle routing problem and test the resulting plans on standard benchmark instances.

Drawback: Benchmarks use straight-line distances, open roads and one depot.

5. Özdamar, Ekinci and Küçükyazici (2004): "Emergency Logistics Planning in Natural Disasters"

Methodology: A multi-period model for dispatching relief goods with vehicles after a natural disaster, solved with a heuristic method.

Drawback: Demand is an input, with no link from weather to need, and the tool is not interactive.

6. Sheu (2007): "An Emergency Logistics Distribution Approach for Quick Response to Urgent Relief Demand in Disasters"

Methodology: Groups affected areas by urgency with fuzzy clustering, then optimises the distribution to the groups.

Drawback: Assumes that urgency and demand are already known.

Comparative Summary

Table 2.1 Comparison of reviewed works with the proposed system

Work	Problem addressed	Technique	Gap for this project
Dantzig and Ramser (1959)	Truck dispatching from a depot	Route construction for a capacity-limited fleet	One depot, no hazards, no priority
Hart et al. (1968)	Cheapest path in a graph	A* best-first search	Needs a flood-aware cost model
Haghani and Oh (1996)	Relief commodity flows	Network flow with heuristics	Fixed demands, no live hazards
Baker and Ayechew (2003); Prins (2004)	Capacitated vehicle routing	Genetic and evolutionary algorithms	Straight-line distances, open roads
Özdamar et al. (2004)	Relief dispatch over several periods	Heuristic method	No link from weather to need
Sheu (2007)	Relief distribution to urgent areas	Fuzzy clustering and optimisation	Demand assumed known
Chennai Relief Control	**Relief dispatch in Chennai**	**Rules, A* and a genetic algorithm**	**Placeholder depots; closure rule untested**

Research Gap

Three gaps emerge from the review. Dispatch models take demand as a given input, so none of the six works links live weather to the needs of a neighbourhood. The genetic algorithms are tested on benchmark instances with

straight-line distances, open roads and one depot, not on a road network that floods change. And none of them is built around an interface in which each number can be traced back to a rule. Chennai Relief Control addresses all three by joining them: hazard readings become area demands, which meet a road network that floods change, which feeds a genetic dispatch with arrival times, all in one interface. It runs on open data and a laptop, needs no solver licence, and can show the reason behind every number. The contribution is the join, not a new algorithm.

CHAPTER 3
PROPOSED SYSTEM

The proposed system is a pipeline in which one set of hazard readings decides both which areas need help and which roads still work, and the dispatch stage then uses both. It has four inputs (weather, earthquakes, roads and depot data), four stages (hazard model, needs and closures, A* routing and genetic dispatch) and one web application.

Only two stages search for answers. A* finds the fastest open route between two places, and the genetic algorithm decides which truck serves which area and in what order. The rest is written-out arithmetic over data files. The service is 4,736 lines of Python in 21 files with 259 lines of tests, and the web application is 4,166 lines of TypeScript in 28 files plus 709 lines of CSS.

Data: Real, Modelled and Placeholder

The weather, earthquake, road and boundary data are real; the populations and hotspot positions are estimates; and the depots, trucks and stock are invented. Table 3.1 keeps the three apart, because every result later in this report inherits the weakest input it touches.

Table 3.1 Inputs and their status

Input	Source	Status
Rain, wind gusts, sea waves, river flow	Open-Meteo forecast, marine and flood APIs, seven sample points across the city	Real, live
Earthquakes	USGS catalogue, events within 600 km in the last 3 days	Real, live
Roads	OpenStreetMap, 8,044 junctions and 10,474 road segments, cached after the first download	Real
15 Chennai corporation zones	Datameet / Transparent Chennai boundaries (CC BY-SA 2.5 IN)	Real
Avadi, Pallavaram, Chrompet, Tambaram	OpenStreetMap boundaries; Chrompet is an approximate split of Pallavaram	Real, approximate
Zone population (6.48 million in total)	2011 census figure for Chennai grown 1% a year, plus an estimate per suburb	Modelled

Input	Source	Status
Low-lying and soil weights per zone	Author's judgement, one value per zone	Modelled
57 flood hotspots	7 from the 2019 NCCR flood atlas, 6 from Greater Chennai Corporation lists, 44 repeatedly reported in 2015, 2021 and 2023; positions within about 300 m	Compiled, approximate
Four past events and two drills	Round figures per event, kept in one file	Representative
23 depots, 46 trucks, depot stock	Placed at well-known facilities; no real inventory exists as open data	Placeholder

3.1 ARCHITECTURE DIAGRAM

The system follows a layered architecture (Fig 3.1). Each layer reads only from the one before it, so a change of data source or of algorithm stays inside its own layer.

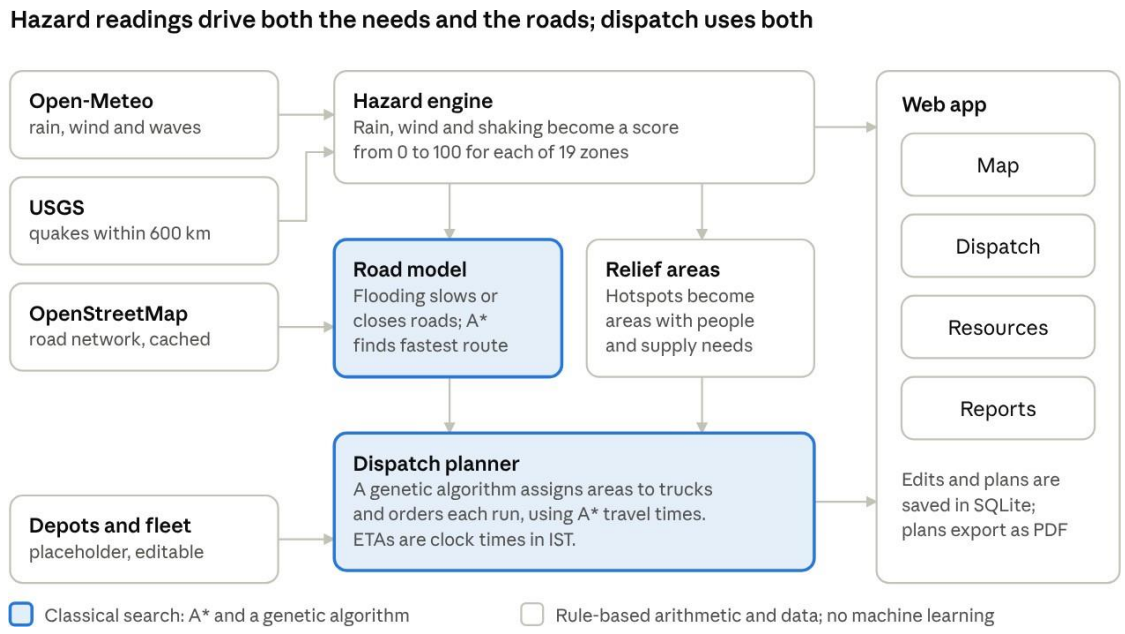


Fig 3.1 System architecture: four inputs, four stages and one web app

- **Input layer.** Open-Meteo supplies rain, wind, sea waves and river flow; the USGS catalogue supplies earthquakes; OpenStreetMap supplies the road network; and the depot, truck and stock tables are edited in the application.

- **Hazard layer.** Written rules turn the readings into a 0 to 100 score for each of the 19 zones and a city-wide disaster meter.
- **Planning layer.** Flood hotspots become areas needing relief, floods slow or close roads, A* finds the fastest open route, and the genetic algorithm assigns trucks and orders the visits.
- **Application layer.** A FastAPI service holds one planning state, and a React application shows it as four pages: Map, Dispatch, Resources and Reports.

CHAPTER 4
MODULES DESCRIPTION

The system is divided into four modules. Module 1 turns readings into hazard scores and the needs of each area. Module 2 builds the flood-aware road model and finds routes with A*. Module 3 assigns trucks and orders the visits with a genetic algorithm. Module 4 presents everything in a web application.

4.1 MODULE 1: HAZARD MODEL AND NEEDS

Each of the 19 zones receives a 0 to 100 score from three written rules, one each for flood, wind and earthquake (Table 4.1), and a city meter summarises them.

Table 4.1 Hazard rules

Hazard	Inputs	Rule
Flood	Rain in 24 h and 72 h, river index, the zone's low-lying value L (0 to 1), hotspot severities in the zone	100 × rain factor × susceptibility × (1 + 0.12 × river × L), where susceptibility = 0.40 + 0.35 L + 0.25 H and H = min(1, total hotspot severity ÷ 9)
Wind	Gust, sea surge, coastal or not	100 × wind factor × (0.60, plus 0.40 if coastal); a surge score of 100 × min(1, surge in m ÷ 2.5) × coastal × (0.7 + 0.3 L) can lift it to 0.9 × the surge score
Earthquake	Magnitude, distance, depth, soil factor	Shaking intensity (MMI) = 1.5 M − 3.3 log10 r + 1, with r the distance and depth combined and at least 5 km; score = 100 × (MMI − 3) ÷ 5 after a soil adjustment

The rain factor is a piecewise-linear curve through the Indian Meteorological Department's 24-hour classes (heavy 64.5 mm, very heavy 115.6 mm, extremely heavy 204.5 mm), and the larger of a 24-hour and a 72-hour curve is used. The wind factor is a similar curve through gust thresholds of 62, 89, 118 and 168 km/h. A curve is a short list of points, so a coordinator can change a threshold by editing one number.

A zone's overall score is its highest hazard score plus 30% of the second highest, capped at 100. The city meter is 0.55 × the worst zone + 0.45 × the

population-weighted average of the zones. Levels start at 25 (Elevated), 50 (High) and 75 (Severe).

Deriving the Needs of an Area

A flood hotspot becomes an area needing relief when its local flood intensity reaches 0.30. Intensity combines the zone's flood score F with the hotspot's recorded severity s (1 to 3):

$$\text{intensity} = \min(1, (F \div 100) \times (0.55 + 0.45 \times s \div 3))$$

The share of the hotspot's exposed residents who are affected then grows faster than linearly with intensity, with a floor of 60 people:

$$\text{share affected} = 0.18 \times ((\text{intensity} - 0.25) \div 0.75)^{1.15}$$

Severity labels come from the same intensity: Critical at 0.78 and above, High at 0.58, Medium at 0.40, otherwise Low. Needs per affected person per day are 1 unit of food, 0.25 of water, 0.07 of medicine, 0.5 of blankets and 0.2 of emergency kits, where a unit is one packaged item. Floods raise water by 30% and food by 10%; earthquakes triple medicine, double kits and raise blankets by 40%; storm damage raises kits by 50% and blankets by 20%. Storm and earthquake damage also create one area per zone when the zone's wind or shaking score reaches 40. At most 60 areas are kept, ranked by risk and by how many people are affected.

4.2 MODULE 2: ROAD NETWORK AND A* ROUTING

The cached road network holds 8,044 junctions and 10,474 road segments. It is not one connected piece: it splits into 153 components, and the largest holds 7,716 junctions (95.9%). A place that snaps to a small component has no route from any depot, so the planner flags it as cut off.

Travel-Time Model

Every road segment has a free-flow truck speed set by its OpenStreetMap class (Table 4.2), and its travel time is length ÷ (speed × flood factor).

Table 4.2 Free-flow truck speed by road class

Road class	Motorway	Trunk	Primary	Secondary	Tertiary	Residential	Link
Speed (km/h)	55	42	34	28	24	18	15

Around every hotspot whose local flood intensity is at least 0.30, each road within the hotspot's radius is slowed by a factor that falls with intensity and rises with distance *d* from the hotspot, never below 0.18:

$$\text{factor} = \max(0.18,\ 1 - 0.85 \times k \times (1 - 0.45 \times d \div r))$$

$$\text{where } k = \min(1,\ (\text{intensity} - 0.2) \div 0.7)$$

Here *r* is the hotspot's radius and *k* measures how far the intensity is above 0.2. A minor road closes outright when intensity is at least 0.82, the hotspot is rated 3, and the road lies within 65% of the hotspot's radius. Motorways, trunk and primary roads never close in this model, and an operator can close or reopen any road by hand from the map.

A* Search

A* expands the junction with the lowest f, which is the travel time so far plus a straight-line estimate of the time left:

$$f(n) = g(n) + h(n), \qquad h(n) = d_{\text{haversine}}(n,\ \text{goal}) \div v_{\max}$$

Here $v_{\max}$ is the fastest speed on any open road. The estimate never overstates the true time, because a road is never shorter than the straight line between its ends and no road is faster than $v_{\max}$. It is also consistent, so a junction never needs to be reopened once expanded, and the route returned is the fastest one given today's flooding.

```
astar(start, goal):
  g[start] <- 0;  open <- queue ordered by f, holding start
  while open is not empty:
     n <- junction in open with the lowest f
     if n == goal: return the path back to start
     for each open road (n, m) with travel time t:
        if g[n] + t < g[m]:
           g[m] <- g[n] + t
           add m to open with f = g[m] + h(m)
  return NOT FOUND            // the place is cut off
```

Because a closed road is simply absent from the graph, the same search serves the dry network and any flooded one. A place with no open route is reported as not found rather than given an invented path.

4.3 MODULE 3: DISPATCH BY GENETIC ALGORITHM

A genetic algorithm decides which truck serves which area and in what order. It evolves 100 candidate plans for 250 generations and keeps the best one it has seen, using only selection, crossover and mutation.

How a Plan Is Encoded

A candidate plan maps each truck to an ordered list of stops. Every area is a stop, except that an area needing more than 5,000 units is split into several deliveries so that any truck can carry one; this turns the 57 areas of the 2015 replay into 69 stops. Each stop sits in exactly one truck's list, and both crossover and mutation keep it that way (Fig 4.1).

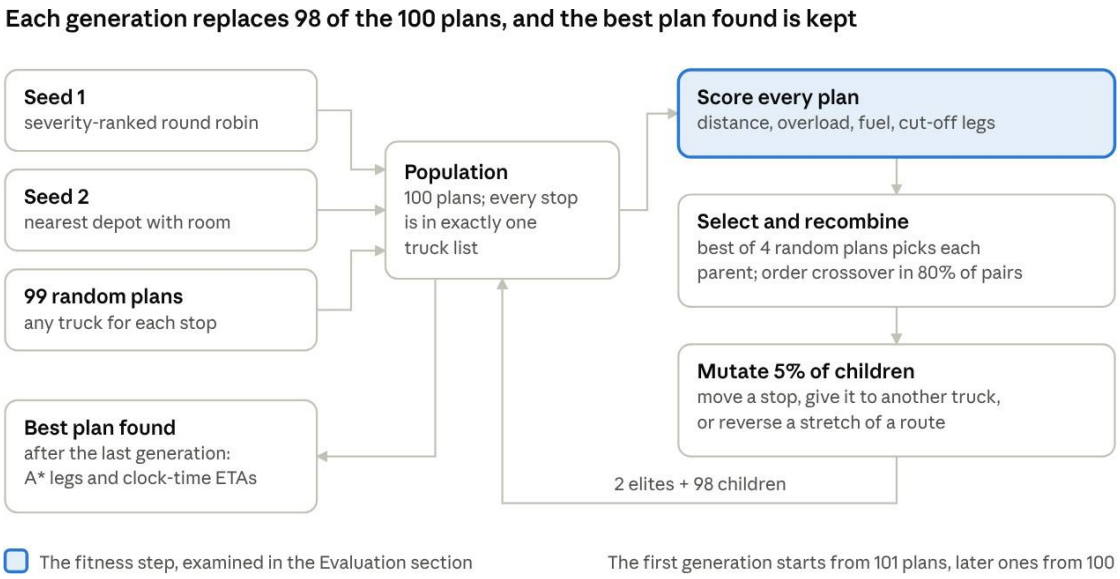


Fig 4.1 Genetic algorithm loop at the default Balanced setting

The first generation starts from 101 plans: two hand-built seeds and 99 random ones; every later generation has 100. One seed deals the stops, most severe first, round-robin across the trucks. The other walks the stops in the same

order and gives each to the nearest of the four closest depots that still has a truck with room, choosing that depot's least-loaded truck.

Operators and Settings

- **Selection.** Tournament of 4, so each parent is the best of four plans drawn at random.
- **Crossover.** Order crossover (OX1) with probability 0.80. A random slice of parent 1 stays where it is, and the remaining stops are filled in parent 2's order, each keeping its truck.
- **Mutation.** 5% of children get one change, chosen at random from three: move a stop to another place or truck, hand a stop to a random truck, or reverse a stretch of one truck's route.
- **Elitism.** The 2 best plans pass to the next generation unchanged.
- **Effort levels in the app.** Fast is 100 generations of 60 plans, Balanced (the default) is 250 of 100, and Best is 600 of 160.

The Fitness Function

Each plan is scored by eight weighted terms, and the highest score wins (Table 4.3).

Table 4.3 Terms of the fitness function

Term	Weight	Meaning	Differs between plans?
Requested demand of visited stops, weighted by severity	+3 × units × weight (Critical 4, High 2.5, Medium 1.5, Low 1)	Serve severe areas	No: every plan visits every stop
Coverage	+2,000 × share of stops visited	Visit all stops	No: always 100%
Unserved stops	−300 each	Do not leave a stop out	No: none is ever left out
Distance	−1.5 per km	Shorter runs	Yes
Overload	−80 per unit above a truck's capacity	Respect truck size	Yes
Fuel overuse	−100 per litre beyond the tank	Respect range	Yes

Term	Weight	Meaning	Differs between plans?
Unreachable leg	−500 each, plus the 9,999 km stand-in distance	Avoid roads that do not exist	Yes
Fleet balance	−10 × spread of stops per truck	Spread work evenly	Yes

Three of the eight terms cannot change the score from one plan to the next. Severity therefore enters the search only through the first seed; after that, the algorithm is a capacity-and-distance optimiser, and the order of stops matters only through distance. The README, the PDF run sheet and a comment in the code describe the fitness as rewarding the most severe areas first, which the code does not do.

From Plan to Timetable

Once the best plan is chosen, every leg is routed with A* on the flood-adjusted network. A truck loads at its depot and serves its stops in order until its capacity or the depot's stock runs out, so a late stop in a run can receive less than it asked for. Travel time scales with the truck's speed against a 36 km/h reference, each stop takes 18 minutes, and every arrival is shown as a clock time in IST from the plan start.

4.4 MODULE 4: WEB APPLICATION

The web application is a React 18 and TypeScript single-page app built with Vite, with MapLibre GL for the map. It reads and writes one planning state held by the FastAPI service. Choosing a scenario on any page changes all four pages, so the map, the plan, the stock tables and the saved reports always describe the same conditions (Table 4.4).

Table 4.4 The four pages of the application

Page	What a coordinator does there	Backed by
Map	Reads the disaster meter and current conditions, sees every zone as a 3D block coloured by risk, finds areas needing relief and depots, opens a card for any depot, area, zone or road, and closes or reopens a road by clicking it	Hazard model, road network
Dispatch	Reads when the first delivery lands, when the critical areas are reached, how many people are reached and what share of need is met; steps through the day on a time bar; replays one A* search as "Explain a route"; downloads PDF run sheets	A*, genetic algorithm
Resources	Adds, edits or removes the 23 depots, 46 trucks, areas needing relief and road closures	Database
Reports	Reopens any earlier plan and downloads its PDF	Saved plans

Choices That Shape the Interface

- **Plain words.** The top-left buttons read Disaster meter, Areas by risk and Layers, and the plan buttons read Plan again and Run sheets (PDF). The interface names what a coordinator is looking for, not how the planner works inside.
- **Always say where the numbers come from.** The top bar shows "Live" with the age of the last reading, "Some live feeds unreachable" with the last good reading, "Live feeds unreachable, showing typical conditions", or "Replaying recorded figures", so a stale input is never mistaken for a current one.
- **Failure states explain themselves.** If the service is not answering after several tries, the page names the service window to check and the one command to start it. Each page sits in its own error boundary, so a fault on one page does not take the others down.
- **Phone and desktop.** Below 760 px the side rail becomes a bottom bar, below 560 px the page title shortens, and the scenario menu is pinned to the screen so it cannot run off either edge. There is also a dark theme.

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The back end was implemented in Python with FastAPI and the front end in React with TypeScript, built with Vite. Table 5.1 lists the environment used for the measurements. The times in this chapter come from a cloud build machine, so a laptop will give different values.

Table 5.1 Hardware and software environment

Item	Specification
Machine	Cloud build machine (Linux)
Back end	Python 3.13, FastAPI; 4,736 lines in 21 files
Front end	React 18, TypeScript, Vite, MapLibre GL; 4,166 lines in 28 files
Testing	Pytest (14 tests); headless Chromium for layout checks
Road data	OpenStreetMap: 8,044 junctions, 10,474 segments
Scenarios	Four past events and two drills (Tables 1.1 and 5.4)

Four kinds of experiment were run:

1. **Correctness.** The 14 tests that ship with the project, run on a clean copy with the network switched off.
2. **Model checks.** Sweeps of rain, gusts and magnitude to confirm that the meter never falls as danger rises; 1,200 depot-to-area routes checked against an independent Dijkstra search; and the effect of floods on 300 routes per event.
3. **Optimiser strength.** The genetic algorithm compared with three simple baselines on all six events, described below.
4. **Interface.** All four pages at six widths, response times, and the time to make a plan.

Design of the Optimiser Comparison

- **Events and inputs.** The six replays, with the same depots, trucks and stock, up to 60 areas, and the default setting of 100 plans over 250 generations. Every plan was made through the app's own planning call, so each figure includes the whole pipeline: needs, closures, A* legs, stock limits and arrival times.
- **Strategies.** (1) The optimiser as shipped. (2) Severity-ranked round robin, the first seed. (3) Nearest depot, the second seed. (4) Random search: 25,000 random plans scored by the same rule as the clamp-free optimiser, best one kept; the optimiser scores about as many plans in one run. (5) The optimiser with the clamp removed.
- **Runs.** The optimiser and random search ran three times per event with different random seeds. The two hand-built plans have no randomness, so each ran once.
- **Measures.** *Reported coverage* is the share of requested relief that the plan says it delivers, the figure on the Dispatch screen. *Road coverage* counts a stop only if every leg of its truck's run up to that stop found an open road. The tables also give Critical areas reached, areas cut off, distance (a leg with no road counts as 0 km, as in the app) and *weighted arrival*: the mean hours to the first delivery by road, weighted by people affected and severity, over the areas reached.

5.2 RESULTS

Correctness

All 14 tests that ship with the project pass, in 1.8 seconds. Table 5.2 summarises them. They show that the pieces work together; they do not show that the optimiser improves anything.

Table 5.2 Automated tests shipped with the project

Group	Tests	What is checked
API	8	The health report says no machine learning is used; all 15 zones, 4 suburbs and at least 50 hotspots load; a replay's state and hazards are served; the earthquake drill scores shaking; a plan has routes, non-decreasing arrival times and a PDF; an A* explanation returns its trace; closing a road changes the route; an area can be added, edited and deleted
A*	4	Straight-line distance; the shortest path on a small graph; blocked roads are avoided; a disconnected target is reported as not found
Genetic algorithm	2	A run on 3 areas and 2 trucks finishes with a plan; order crossover returns every stop exactly once

Table 5.3 lists the checks made for this report on top of the shipped tests.

Table 5.3 Checks made for this report

Check	Method	Result
A* is optimal	1,200 depot-to-area pairs over three replays, each solved again by a separate Dijkstra search	1,008 routes found by both, all at the same cost to within 0.0001 h; the other 192 pairs had no route in either
Meter never falls as danger rises	Rain 0 to 375 mm in 16 steps, gusts 0 to 180 km/h in 19 steps, magnitude 4.0 to 7.5 in 8 steps	The city meter did not fall at any step
No sideways scrolling	All four pages at 320, 390, 768, 1,024, 1,440 and 1,920 px wide, measured in a browser	0 px of horizontal overflow in all 24 cases
Response time	Median of 15 requests for each endpoint	State 16 ms (68 KB), zone shapes 9 ms (46 KB), scenario list 2.8 ms, hazards 2.4 ms
Scoring cost	Scoring all 19 zones, and re-applying a flood to the whole road network	0.16 ms and 17 ms
Plan time	Default setting (100 plans, 250 generations), 57 to 60 areas	First plan for a new event 41 to 51 s, because each route is searched the first time it is needed; later plans 7 to 12 s

The genetic algorithm test asserts only that the best fitness is above zero and that the history has 100 entries. Because the score is clamped to a minimum of 0.01 (Finding 1 below), a search that never moves also passes, and the toy case has three areas and no cut-off legs. A regression test needs a map with at least one cut-off leg and an assertion that the final best score is above the best of the starting plans.

Hazard Model Results

Fig 5.1 sweeps rain alone from 0 to 375 mm. Rain lifts the meter from 0 to 87.1, and the curve stops there because the rain factor reaches 1.0 at 300 mm, so 300 mm and 375 mm look the same to the model.

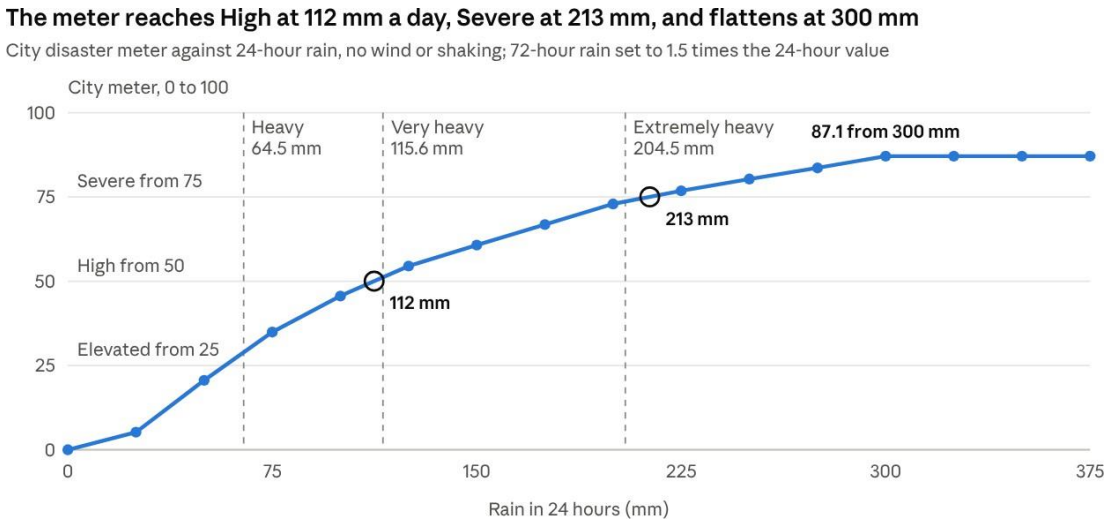


Fig 5.1 Disaster meter against 24-hour rain, with no wind or shaking

Table 5.4 gives what the six replays produce. The model puts Michaung (96.4) above the 2015 floods (94.1) although its replay rain is smaller. Both have a worst zone at 100, but Michaung's gusts and surge add to the flood score in most zones and lift the population-weighted average from 87.0 to 91.9. The order is a property of the representative inputs, not a verdict on the real events. The 60-area cap also matters: four of the six replays reach it, so their people and needs cover the 60 highest-ranked areas only.

Table 5.4 What the six events produce

Event	24 h rain, gust	Meter	Zones High or above	Areas flagged	People affected	Items needed	Roads slowed / closed
Floods 2015	345 mm, 45 km/h	94.1	19 of 19	57 (28 Critical)	87,060	191,179	1,893 / 219
Michaung 2023	290 mm, 95 km/h	96.4	19 of 19	60 (25 Critical)	118,660	261,489	1,903 / 209
Fengal 2024	140 mm, 80 km/h	75.4	18 of 19	60 (none Critical)	55,090	121,448	2,112 / 0
Vardah 2016	90 mm, 140 km/h	90.1	19 of 19	60 (7 Critical)	146,560	325,109	1,537 / 0
Cyclone drill	190 mm, 150 km/h	94.7	19 of 19	60 (13 Critical)	198,370	439,175	2,112 / 0
Earthquake drill (M6.6)	none	63.7	17 of 19	19 (none Critical)	138,200	353,807	0 / 0

Routing Results

For three replays, 400 random depot-to-area pairs were solved by the project's A* and by a separate Dijkstra search written for the test (Table 5.5). Wherever a route existed, the two travel times agreed to within the 0.0001 h rounding of the A* result, and no pair was reachable by one and not the other.

Table 5.5 A* compared with Dijkstra

Event	Routes found	Same cost	Junctions expanded, A* vs Dijkstra	Time per route, A* vs Dijkstra
Floods 2015	310	310	2,935 vs 4,142 (29% fewer)	10.2 vs 7.4 ms
Michaung 2023	307	307	2,906 vs 4,154 (30% fewer)	12.3 vs 9.0 ms
Cyclone drill	391	391	2,780 vs 4,118 (33% fewer)	9.6 vs 7.9 ms

A* looks at about 30% fewer junctions but is 20 to 40% slower per route in this Python code, because each junction it looks at costs a trigonometric distance. The estimate is also loose: it assumes the fastest road, while most roads run at 18 to 28 km/h. A* is therefore chosen here for its step-by-step trace, which the Dispatch page replays as "Explain a route", not for speed.

The same 300 random pairs per event were then routed twice, once on the dry network and once under the replayed flood (Table 5.6).

Table 5.6 What floods do to routes

Event	No open road under flood	Dry-weather route now closed	Best route changes	Mean extra travel time
Floods 2015	54 of 300 (18%)	37 of 246 (15%)	67%	+19.9%
Michaung 2023	66 of 300 (22%)	31 of 234 (13%)	65%	+17.2%
Cyclone drill	8 of 300 (3%)	0 of 292	49%	+11.7%

A planner that ignored floods would send a truck into a closed road on about one route in seven (13% and 15% in the two flood replays) and would underestimate travel times by roughly 15 to 17%. Even where the dry-weather route stays open it is slower than the flood-aware one by 1.9% on average and by up to 17.9% in the 2015 replay.

Optimiser Results

As shipped, the optimiser never improves on its first plan. With one line changed it delivers more relief by road than three simple baselines on five of the six events and ties on the sixth. The headline numbers in the app are higher than what trucks can reach by road, by the most in the two flood replays.

Finding 1: the shipped optimiser never improves on its first plan

On all six events and in all three runs, the plan the app returns is the severity-ranked round robin: coverage and distance match to the last digit, and the best score is the same in the first generation as in the last. The cause is the clamp. Every starting plan in the 2015 replay has a raw score of about −10 to −12 million. About 35 legs have no road, and each is charged 9,999 km of distance and thousands of litres of fuel against tanks of 60 to 200 litres; the trucks are also 52,000 to 81,000 units over capacity. The code then sets the score to max(raw, 0.01), so every plan scores exactly 0.01 (Table 5.7). Selection compares equal numbers, crossover and mutation cannot be judged, and elitism keeps the first two plans, which are the seeds.

Table 5.7 Starting plans and final plan in the 2015 replay

2015 replay	Distance counted (km)	Cut-off legs	Units over capacity	Score after clamp
Seed 1, severity-ranked	351,433	35	63,976	0.01
Seed 2, nearest depot	350,420	35	52,315	0.01
A random starting plan	341,471	34	80,787	0.01
The optimiser's final plan	351,433	35	63,976	0.01

Finding 2: the screen reports more reach than the roads give

The plan builder records a leg that found no road as 0 km and 0 hours, so the stop after it gets an arrival time and its units count as delivered. Table 5.8 shows the effect. The gap is widest where floods close roads: 23.5 points in the 2015 replay and 20.7 in Michaung, against 4.8 to 7.4 in the other four. In the 2015 plan the screen says 39,090 people are reached; by road the figure is 26,160. The cut-off areas are not hidden, since the amber panel names nine of them, but the headline and the first-delivery tile ignore them.

Table 5.8 Reported coverage against road coverage (shipped optimiser)

Event	Reported coverage	Road coverage	Critical areas reached	Areas cut off	Legs with no road
Floods 2015	66.5%	43.0%	19 of 28	24 of 57	30 of 69
Michaung 2023	58.1%	37.4%	16 of 25	21 of 60	32 of 83
Fengal 2024	83.9%	79.1%	none Critical	2 of 60	3 of 66
Vardah 2016	49.7%	42.3%	6 of 7	7 of 60	6 of 109
Cyclone drill	37.7%	31.5%	12 of 13	6 of 60	7 of 118
Earthquake drill	45.9%	39.4%	none Critical	2 of 19	5 of 80

How the optimiser compares with simple plans

With the clamp removed, the optimiser has the highest road coverage on five events (Fig 5.2, Table 5.9). In all 15 of its runs on those five it beats every run of every baseline, by 2.5 to 17.4 points over the best baseline's mean: 17.4 on the 2015 floods and 13.4 on Michaung, then 9.4, 3.3 and 2.5. On the cyclone drill it ties random search, 35.7 against 35.2, and the run ranges overlap. Where no road closes, as in Fengal, sending each stop from the nearest depot already beats the shipped optimiser by 5.9 points.

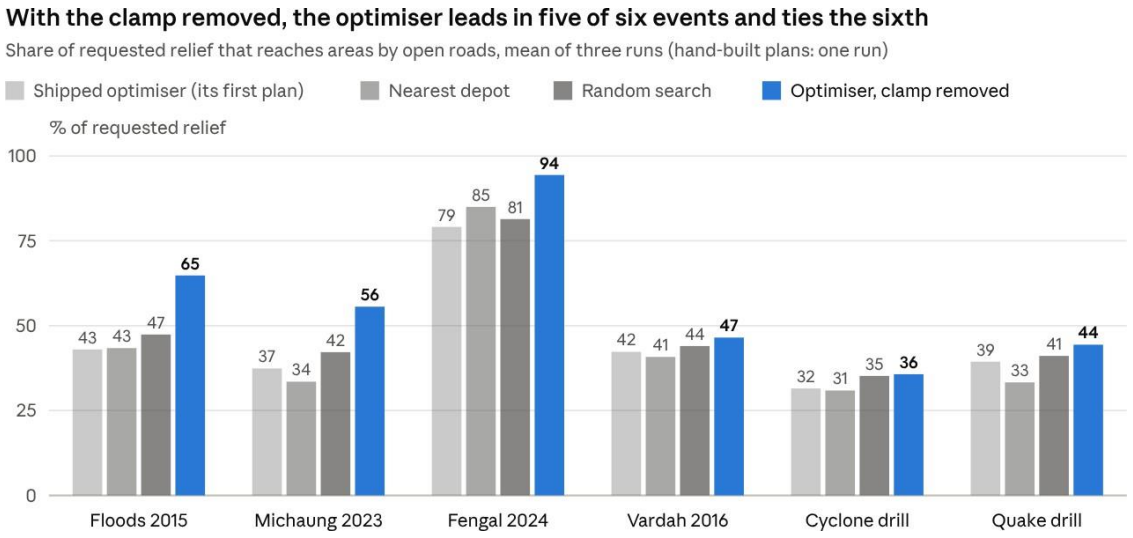


Fig 5.2 Road coverage by strategy and event (mean of three runs)

Table 5.9 Road coverage, % (range over three runs)

Event	Shipped optimiser	Nearest depot	Random search	Optimiser, clamp removed
Floods 2015	43.0	43.4	47.4 (45.6 to 49.2)	64.8 (64.1 to 65.7)
Michaung 2023	37.4	33.5	42.2 (40.2 to 45.0)	55.6 (54.7 to 57.3)
Fengal 2024	79.1	85.0	81.4 (78.9 to 83.2)	94.4 (94.1 to 94.7)
Vardah 2016	42.3	40.8	44.0 (43.5 to 44.5)	46.5 (45.6 to 47.2)
Cyclone drill	31.5	30.9	35.2 (34.7 to 36.1)	35.7 (35.4 to 36.0)
Earthquake drill	39.4	33.3	41.1 (40.2 to 41.8)	44.4 (42.7 to 46.0)

The 2015 flood replay in detail

Table 5.10 The 2015 flood replay by strategy

Strategy	Reported coverage	Road coverage	Critical reached (of 28)	Cut off (of 57)	Distance (km)	Weighted arrival (h)
Shipped optimiser (the severity-ranked plan)	66.5%	43.0%	19	24	1,468	0.84
Nearest depot	72.6%	43.4%	18	17	454	0.49
Random search	64.4%	47.4%	17.3	13.7	1,589	1.19
Optimiser, clamp removed	72.8%	64.8%	19	9	1,549	0.93
Clamp removed and cut-off legs counted as free	82.9%	51.7%	19	14.7	1,003	0.65

The nearest-depot plan arrives first, at 0.49 h, and covers less than a third of the distance, but it leaves 8 more areas cut off than the clamp-free optimiser and delivers 21.4 points less by road. Distances are not a fair comparison across

rows: a leg with no road counts as 0 km, so plans with more cut-off legs look shorter. The clamp-free plan arrives a little later than the shipped one, 0.93 h against 0.84 h, partly because its average covers 15 more reached areas.

Why the one-line change works, and a repair that backfires

Without the clamp, each cut-off leg costs about 15,000 points of distance plus a fuel penalty, which dwarfs everything else, so the search spends its first generations removing cut-off legs (Fig 5.3). Reported coverage rises only 6.3 points on the 2015 replay, from 66.5% to 72.8%, while road coverage rises 21.8 points, from 43.0% to 64.8%.

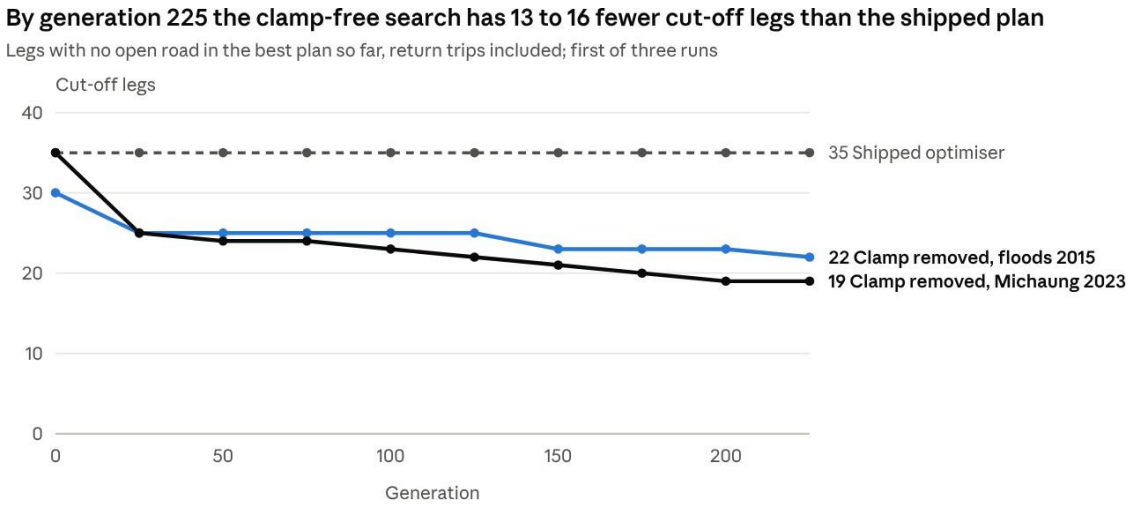


Fig 5.3 Cut-off legs in the best plan, by generation

The 9,999 km stand-in looks like a bug, and the obvious repair is to count a cut-off leg as 0 km, as the plan builder does. That was tested as a second change. On the 2015 replay it gives the highest reported coverage of any variant, 82.9%, and a road coverage of 51.7%, which is 13.1 points below the one-line change. It loses 7.4 points on Michaung and 6.7 on Fengal, and ties within 0.3 points on the other three events. The search had learned to raise the number on the screen. The stand-in distance is the only thing in the score that teaches the search about missing roads, so the real repair also makes the plan builder and the headline honest about cut-off legs.

How much search is worth it

Table 5.11 repeats the clamp-free optimiser at the app's three effort levels, with three runs each. These runs used different random seeds from the comparison above, so the default-setting figures differ slightly from it. Moving from Fast to Balanced adds 3.7 to 6.5 points. Moving from Balanced to Best adds 0.7 to 3.1 points for nearly four times the scoring work, so the default is a reasonable stopping point. On Michaung the three runs at the default setting span 6 points, so differences of a few points on that event are within the noise.

Table 5.11 Road coverage, % (range over three runs), by effort level

Effort	Plans × generations	Plans scored	Floods 2015	Michaung 2023	Fengal 2024
Fast	60 × 100	6,000	57.8 (56.7 to 59.7)	47.1 (44.9 to 48.5)	90.3 (89.6 to 91.8)
Balanced (default)	100 × 250	25,000	64.3 (64.2 to 64.6)	53.0 (48.9 to 55.1)	94.0 (93.4 to 94.7)
Best	160 × 600	96,000	66.5 (66.0 to 67.5)	56.1 (55.0 to 57.0)	94.7 (94.5 to 95.0)

The change to the project

The one-line change is in `backend/algorithms/genetic.py`, line 318: `fitness = max(raw_fitness, 0.01)` becomes `fitness = raw_fitness`. In a scratch copy all 14 project tests still pass with it. The only visible side effect is that the stored score history becomes negative; nothing in the app or the PDF displays those values. The project's own files have not been touched by this report.

Output Screenshots

Figs 5.4 to 5.6 show the application replaying the 2015 floods.

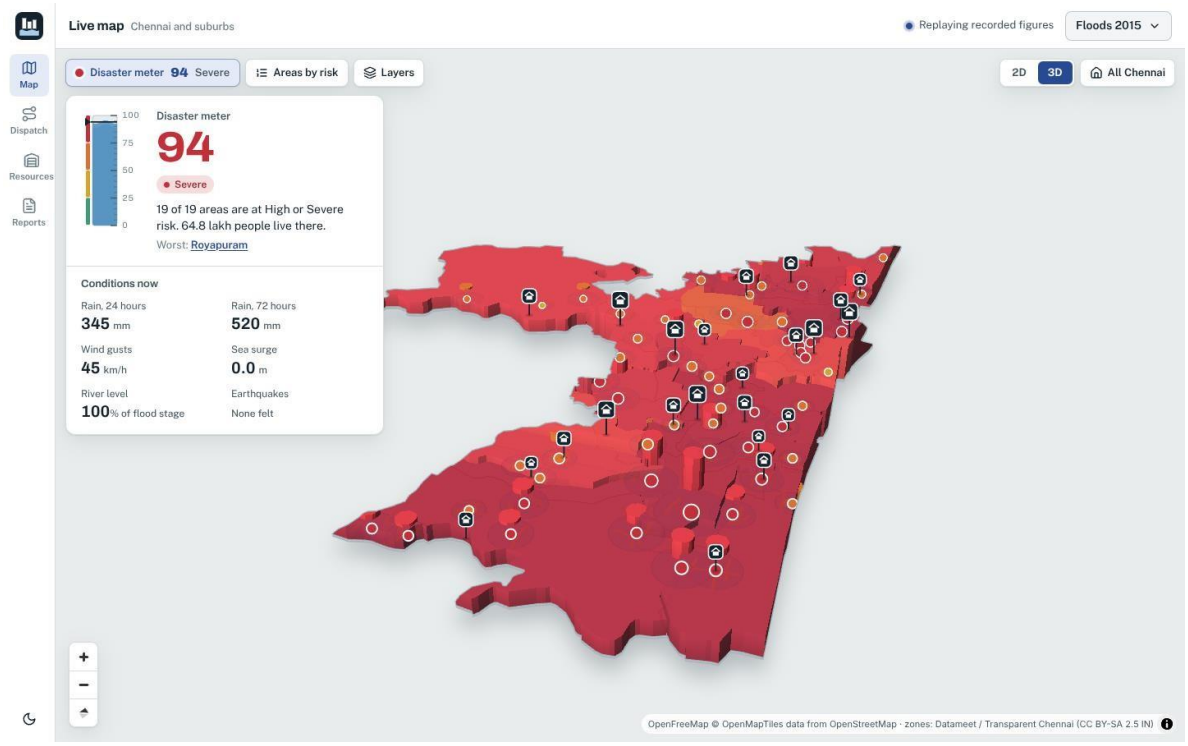


Fig 5.4 Map page: zones as 3D blocks coloured by risk, depots and areas needing relief

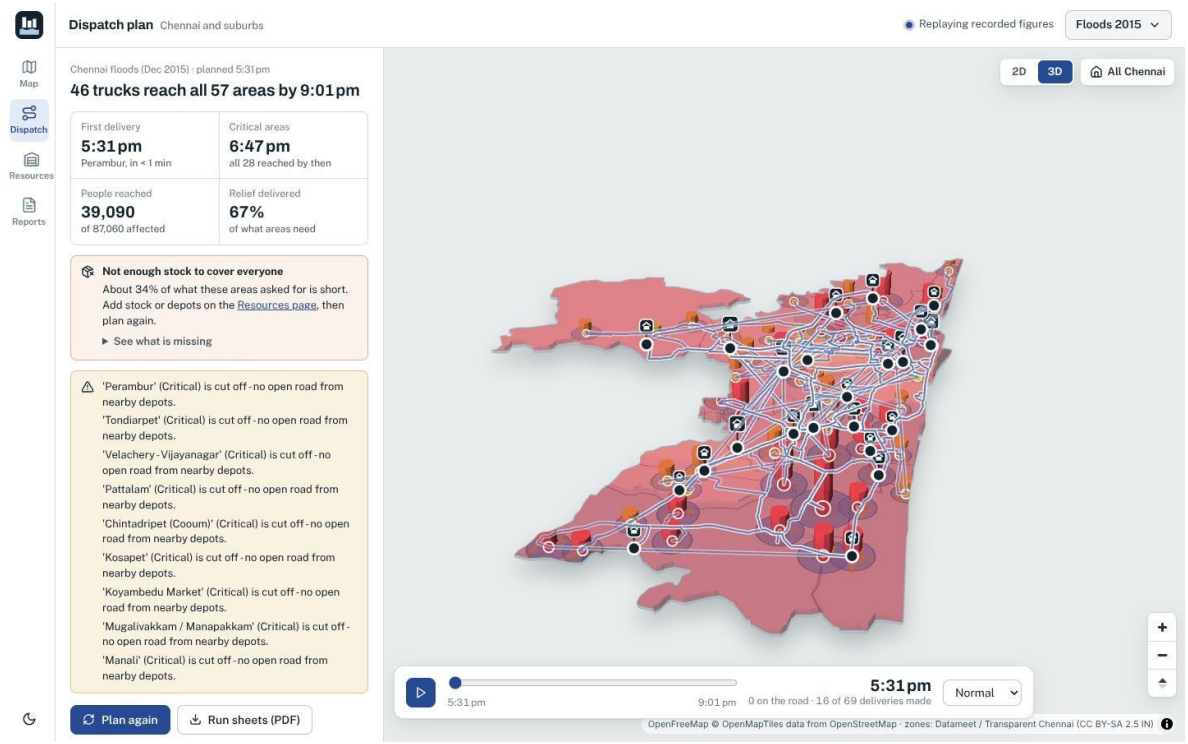


Fig 5.5 Dispatch page at the default setting

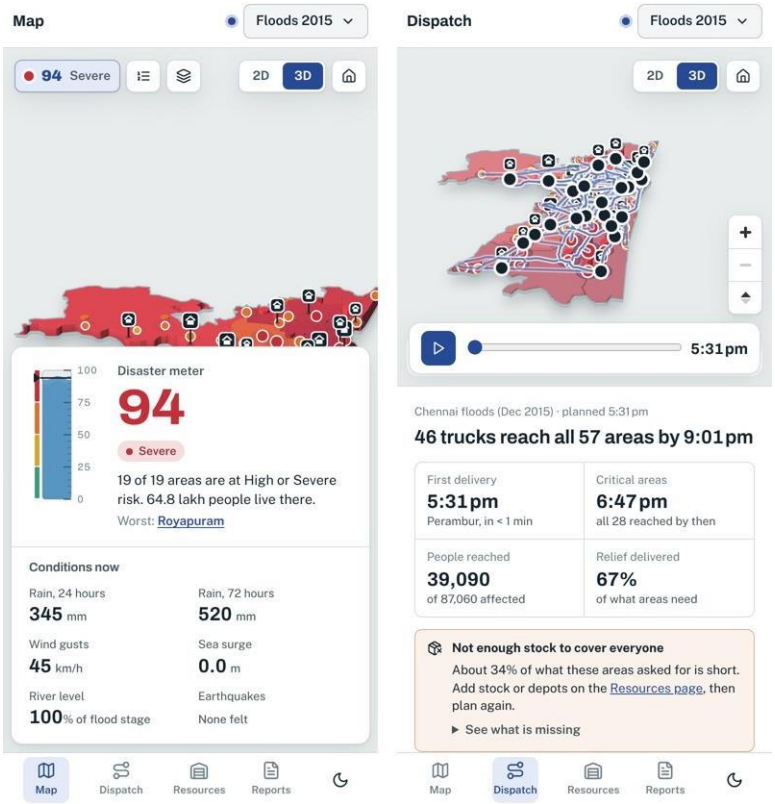


Fig 5.6 Responsive layout on a 390 px wide phone

The street basemap did not load on the machine that took the captures, so the map background is plain. In Fig 5.5 the headline reads “46 trucks reach all 57 areas”, yet the amber panel below it lists Critical areas as cut off. The first-delivery tile reads 5:31 pm at Perambur, “in < 1 min”, although the same panel lists Perambur as cut off: the leg from the Koyambedu depot to Perambur has no road, and the plan builder records it as 0 km and 0 hours. The panel lists nine Critical areas as cut off, while the tile above it says all 28 Critical areas are reached by 6:47 pm.

CHAPTER 6
CONCLUSION AND FUTURE WORK

Conclusion

The project delivers what its objectives asked for on open data and without machine learning. Live or replayed weather becomes a zone score and a list of areas needing relief, floods slow and close roads, A* finds the fastest open route, and a genetic algorithm assigns trucks and visiting order with clock-time arrivals, all in a web application that works on a phone. The parts built on written rules held up under test: A* matched an independent Dijkstra search on 1,008 routes, the disaster meter never fell as danger rose, and no page scrolls sideways at any of the six widths tested.

The evaluation also found two faults that the shipped tests could not see. The optimiser does not optimise, because a clamp on its score makes every plan equal; and the Dispatch screen counts cut-off areas as reached, so on the 2015 replay it reports 67% delivered when 43% reaches an area by road. Removing one line lifts road coverage by 21.8 points on that replay and leads every baseline on five of six events, but the screen would still overstate the result. Everything here was measured on stand-in depots, trucks and stock, so the numbers show how the planner behaves, not how a real relief operation would go.

Limitations

- **Invented inputs.** The 23 depots, 46 trucks and 630,240 units of stock are not real inventories. In the 2015 replay the fleet can carry 166,700 units in one trip against 191,179 requested, so no plan can deliver more than 87% of the request.
- **Round replay figures and modelled people.** Each event is one rain total and one peak gust for the whole city, and people affected come from a zone estimate and a formula, not from a count. The 57 hotspot positions are within about 300 m.

- **An untested closure rule.** Nobody has compared the modelled closures with the roads that actually flooded, so the share of cut-off places is an output of the rule, not an observation. In the 2015 replay, 399 of 1,587 depot-to-stop pairs (25%) have no open route.
- **Gaps in the road network and the model.** 4.1% of junctions lie outside the largest connected piece, so some places are cut off even on a dry day. Rain saturates at 300 mm, plans have one trip per truck and no live traffic, and at most 60 areas are kept.
- **A score that cannot see order or time.** Severity enters only through the first seed, and nothing rewards reaching a Critical area earlier than a Low one. The documentation says otherwise.
- **Small samples.** Three runs per setting and one input per event show direction and rough size, not exact figures.

Responsible Use

A plan from this tool is a proposal for a coordinator to read, check and change, not an instruction. Before it informs anything real, the placeholder depots, fleet and stock must be replaced with real ones; the cut-off list must be checked against local knowledge of which roads are passable; and the freshness line in the top bar must be read as part of the data. The application works at neighbourhood level and stores no names or contact details of residents. Place photos come from Wikimedia with the photographer and licence shown.

Future Work, in Order of Value

1. **Remove the clamp and add a regression test.** One line, plus a test on a map with at least one cut-off leg that asserts the final score beats the best starting plan. Evidence: 2.5 to 17.4 points more road coverage on five of six events.
2. **Make the screen honest about cut-off areas.** Leave them out of coverage, people reached and arrival times, and show “no open road” where the first-

delivery tile now shows a time. Evidence: 67% reported against 43% by road on the 2015 replay.

3. **Teach the score about severity and time.** Add a penalty for each hour a Critical area waits, weighted by the people in it.
4. **Correct the documentation.** The README, the PDF run sheet and a code comment say the algorithm rewards covering the most severe areas first; describe the score as it is, or make the previous step true.
5. **Use real inputs and check the closure rule.** Replace the placeholder depots, trucks and stock, compare the model's closures with where roads actually flooded in 2015 and 2023, then repeat this evaluation.
6. **Later.** Multi-trip schedules, live traffic, and a study of how sensitive the plans are to the closure thresholds.

CHAPTER 7
REFERENCES

Altay, N. and Green, W.G. (2006) 'OR/MS research in disaster operations management', *European Journal of Operational Research*, Vol.175, No.1, pp.475-493.

Baker, B.M. and Ayechew, M.A. (2003) 'A genetic algorithm for the vehicle routing problem', *Computers & Operations Research*, Vol.30, No.5, pp.787-800.

Dantzig, G.B. and Ramser, J.H. (1959) 'The truck dispatching problem', *Management Science*, Vol.6, No.1, pp.80-91.

Datameet (n.d.) *Community maps and open data for India*, available at: https://datameet.org/ (Chennai corporation zone boundaries, CC BY-SA 2.5 IN).

Haghani, A. and Oh, S.-C. (1996) 'Formulation and solution of a multi-commodity, multi-modal network flow model for disaster relief operations', *Transportation Research Part A*, Vol.30, No.3, pp.231-250.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics*, Vol.4, No.2, pp.100-107.

India Meteorological Department (n.d.) *Rainfall and wind warning classes*, available at: https://mausam.imd.gov.in/ (thresholds only; no data is copied).

Open-Meteo (2026) *Open-Meteo Weather, Marine and Flood APIs*, available at: https://open-meteo.com/

OpenFreeMap (2026) *OpenFreeMap street basemap*, available at: https://openfreemap.org/

OpenStreetMap contributors (2026) *OpenStreetMap*, available at: https://www.openstreetmap.org/ (road network, ODbL).

Özdamar, L., Ekinci, E. and Küçükyazici, B. (2004) 'Emergency logistics planning in natural disasters', *Annals of Operations Research*, Vol.129, pp.217-245.

Prins, C. (2004) ‘A simple and effective evolutionary algorithm for the vehicle routing problem’, *Computers & Operations Research*, Vol.31, No.12, pp.1985-2002.

Sheu, J.-B. (2007) ‘An emergency logistics distribution approach for quick response to urgent relief demand in disasters’, *Transportation Research Part E*, Vol.43, No.6, pp.687-709.

The News Minute (n.d.) *How grave lapses at Chembarambakkam lake caused Chennai’s manmade flood*, available at: https://thenewsminute.com/article/how-grave-lapses-chembarambakkam-lake-caused-chennai-s-manmade-flood-2015-84524

Transparent Chennai (n.d.) *Chennai ward and zone boundaries*, available at: https://www.transparentchennai.com/

USGS (2026) *Earthquake Catalog*, available at: https://earthquake.usgs.gov/

Wikimedia Commons (2026) *Place photographs shown in the application, with photographer and licence*, available at: https://commons.wikimedia.org/

Wikipedia (2026a) *2015 South India floods*, available at: https://en.wikipedia.org/wiki/2015_South_India_floods

Wikipedia (2026b) *Cyclone Vardah*, available at: https://en.wikipedia.org/wiki/Cyclone_Vardah

Wikipedia (2026c) *Cyclone Michaung*, available at: https://en.wikipedia.org/wiki/Cyclone_Michaung

Wikipedia (2026d) *Cyclone Fengal*, available at: https://en.wikipedia.org/wiki/Cyclone_Fengal